MODUL BELAJAR PYTHON

String & Regular Expression

Mencakup 5 Topik Utama:

1. Pengenalan String
2. Mengakses & Memanipulasi String
3. Operator & Metode String
4. Parsing String
5. Regular Expression (Regex)

BAB 1: Pengenalan String

1.1 Apa itu String?

String adalah tipe data yang digunakan untuk menyimpan teks (urutan karakter). Dalam Python, string ditulis di antara tanda kutip tunggal (' ') atau ganda (" ").

> **Definisi**
> String = kumpulan karakter yang berurutan. Contoh: 'Halo', "Python", '123abc'.

1.2 Cara Membuat String

```
# Kutip tunggal
teks1 = 'Halo Dunia'

# Kutip ganda
```

```
teks2 = "Belajar Python"

# Multi-line string (triple quotes)
teks3 = '''Ini adalah
string yang panjang
lebih dari satu baris'''

# String kosong
teks4 = ''

print(teks1)
print(teks2)
print(teks3)
print(type(teks1))  # cek tipe data
```

Output:

```
Halo Dunia
Belajar Python
Ini adalah
string yang panjang
lebih dari satu baris
<class 'str'>
```

1.3 Escape Character

Escape character digunakan untuk menyisipkan karakter khusus dalam string menggunakan backslash (\).

```
# \n = new line (baris baru)
print('Baris 1\nBaris 2')

# \t = tab
print('Nama:\tBudi')

# \\ = backslash
print('C:\\Users\\Budi')

# \' = kutip tunggal
print('Ini adalah \'contoh\' escape')
```

Output:

```
Baris 1
Baris 2
Nama:   Budi
C:\Users\Budi
```

```
Ini adalah 'contoh' escape
```

BAB 2: Mengakses & Memanipulasi String

2.1 Indexing (Mengakses Karakter)

Setiap karakter dalam string memiliki indeks yang dimulai dari 0. Python juga mendukung indeks negatif (dari belakang).

Cara Kerja Indeks

teks = 'Python' Indeks: P=0, y=1, t=2, h=3, o=4, n=5 Indeks negatif: P=-6, y=-5, t=-4, h=-3, o=-2, n=-1

```
teks = 'Python'

# Indeks positif (dari depan)
print(teks[0])   # P
print(teks[1])   # y
print(teks[5])   # n

# Indeks negatif (dari belakang)
print(teks[-1])  # n
print(teks[-2])  # o
print(teks[-6])  # P
```

Output:

```
P
y
n
n
o
P
```

2.2 Slicing (Memotong String)

Slicing digunakan untuk mengambil sebagian dari string. Sintaks: teks[start : stop : step]

```
teks = 'Belajar Python'
```

```
print(teks[0:7])     # 'Belajar'     (dari 0 s/d 6)
print(teks[8:])      # 'Python'      (dari 8 sampai akhir)
print(teks[:7])      # 'Belajar'     (dari awal s/d 6)
print(teks[-6:])     # 'Python'      (6 karakter terakhir)
print(teks[::2])     # setiap 2 karakter
print(teks[::-1])    # string dibalik
```

Output:

```
Belajar
Python
Belajar
Python
Blja yhn
nohtyP rajaleBB
```

2.3 Manipulasi Dasar

```
nama = 'Budi Santoso'

# Panjang string
print(len(nama))            # 12

# Huruf besar / kecil
print(nama.upper())         # BUDI SANTOSO
print(nama.lower())         # budi santoso
print(nama.title())         # Budi Santoso
print(nama.swapcase())      # bUDI sANTOSO

# Menghapus spasi di tepi
teks = '  Halo  '
print(teks.strip())         # 'Halo'
print(teks.lstrip())        # 'Halo  '
print(teks.rstrip())        # '  Halo'
```

Output:

```
12
BUDI SANTOSO
budi santoso
Budi Santoso
bUDI sANTOSO
Halo
Halo
  Halo
```

BAB 3: Operator & Metode String

3.1 Operator String

```
a = 'Hello'
b = ' World'

# Penggabungan (+)
gabung = a + b
print(gabung)            # Hello World

# Pengulangan (*)
ulang = 'Ha' * 3
print(ulang)             # HaHaHa

# Keanggotaan (in / not in)
print('ell' in a)        # True
print('xyz' in a)        # False
print('xyz' not in a)    # True

# Perbandingan
print('abc' == 'abc')    # True
print('abc' < 'abd')     # True (bandingkan huruf per huruf)
```

Output:

```
Hello World
HaHaHa
True
False
True
True
True
```

3.2 Tabel Metode String Penting

Metode	Fungsi	Contoh
`find()`	Mencari posisi substring	`"hello".find("ll") → 2`
`index()`	Seperti find, tapi error jika tidak ada	`"hello".index("e") → 1`
`count()`	Menghitung kemunculan substring	`"banana".count("a") → 3`

`replace()`	Mengganti substring	`"hai".replace("i","lo") → "halo"`
`split()`	Memecah string jadi list	`"a,b,c".split(",") → [a,b,c]`
`join()`	Menggabungkan list menjadi string	`",".join(["a","b"]) → "a,b"`
`startswith()`	Cek awalan string	`"Python".startswith("Py") → True`
`endswith()`	Cek akhiran string	`"data.csv".endswith(".csv") → True`
`isdigit()`	Cek apakah semua digit	`"123".isdigit() → True`
`isalpha()`	Cek apakah semua huruf	`"abc".isalpha() → True`
`isalnum()`	Cek huruf dan/atau angka	`"abc123".isalnum() → True`
`center()`	Posisi string di tengah	`"Hi".center(10,"-") → "----Hi----"`
`zfill()`	Isi kiri dengan nol	`"42".zfill(5) → "00042"`

3.3 Contoh Penggunaan Metode

```
kalimat = 'belajar python itu menyenangkan'

# find dan count
print(kalimat.find('python'))      # 8
print(kalimat.count('a'))          # 4

# replace
baru = kalimat.replace('python', 'Java')
print(baru)

# split dan join
kata_kata = kalimat.split(' ')
print(kata_kata)
print(', '.join(kata_kata))

# startswith dan endswith
print(kalimat.startswith('belajar'))     # True
print(kalimat.endswith('kan'))           # True
```

Output:

```
8
4
belajar Java itu menyenangkan
['belajar', 'python', 'itu', 'menyenangkan']
```

```
belajar, python, itu, menyenangkan
True
True
```

3.4 String Formatting

```
nama = 'Siti'
usia = 20
nilai = 95.5

# f-string (Python 3.6+) - DIREKOMENDASIKAN
print(f'Nama: {nama}, Usia: {usia} tahun')
print(f'Nilai: {nilai:.1f}')

# format()
print('Nama: {}, Usia: {} tahun'.format(nama, usia))

# % operator (cara lama)
print('Nama: %s, Usia: %d' % (nama, usia))

# Format angka
angka = 1234567.89
print(f'{angka:,.2f}')     # format ribuan
print(f'{angka:e}')        # notasi ilmiah
```

Output:

```
Nama: Siti, Usia: 20 tahun
Nilai: 95.5
Nama: Siti, Usia: 20 tahun
Nama: Siti, Usia: 20
1,234,567.89
1.234568e+06
```

BAB 4: Parsing String

4.1 Apa itu Parsing String?

Parsing string adalah proses mengurai atau mengekstrak informasi dari string. Teknik ini sangat berguna saat memproses data teks.

4.2 Parsing dengan split()

```
# Parsing data CSV sederhana
data_csv = 'Budi,25,Jakarta,Programmer'
kolom = data_csv.split(',')
print('Nama:', kolom[0])
print('Usia:', kolom[1])
print('Kota:', kolom[2])
print('Pekerjaan:', kolom[3])

# Parsing dengan maxsplit
teks = 'apple:banana:cherry:date'
hasil = teks.split(':', 2)  # Hanya 2 kali split
print(hasil)
```

Output:

```
Nama: Budi
Usia: 25
Kota: Jakarta
Pekerjaan: Programmer
['apple', 'banana', 'cherry:date']
```

4.3 Parsing Tanggal & Waktu

```
# Parsing tanggal manual
tanggal_str = '15/04/2024'
bagian = tanggal_str.split('/')
hari = int(bagian[0])
bulan = int(bagian[1])
tahun = int(bagian[2])
print(f'Hari: {hari}, Bulan: {bulan}, Tahun: {tahun}')

# Parsing dengan datetime
from datetime import datetime
tanggal = datetime.strptime('15/04/2024', '%d/%m/%Y')
print(tanggal.strftime('%d %B %Y'))
print(tanggal.year)
```

Output:

```
Hari: 15, Bulan: 4, Tahun: 2024
15 April 2024
2024
```

4.4 Parsing Data Terstruktur

```
# Parsing log file sederhana
log = '[ERROR] 2024-04-15 10:30:22 - File not found: data.txt'

# Metode 1: split dan strip
bagian = log.split(']')
level = bagian[0].replace('[', '').strip()
sisa = bagian[1].strip()
print('Level:', level)
print('Sisa info:', sisa)

# Parsing multi-baris
data = '''
Nama: Andi Setiawan
Email: andi@email.com
Telepon: 08123456789
'''

for baris in data.strip().split('\n'):
    if ':' in baris:
        kunci, nilai = baris.split(':', 1)
        print(f'{kunci.strip()} => {nilai.strip()}')
```

Output:

```
Level: ERROR
Sisa info: 2024-04-15 10:30:22 - File not found: data.txt
Nama => Andi Setiawan
Email => andi@email.com
Telepon => 08123456789
```

BAB 5: Regular Expression (Regex)

5.1 Apa itu Regular Expression?

Regular Expression (Regex) adalah pola teks yang digunakan untuk mencari, mencocokkan, dan memanipulasi string secara fleksibel. Python menyediakan modul re untuk regex.

Import Modul

import re Modul re harus diimport terlebih dahulu sebelum menggunakan regex.

5.2 Karakter Spesial Regex

Simbol	Arti
`.`	Karakter apa saja (kecuali newline)
`^`	Awal string
`$`	Akhir string
`*`	0 atau lebih pengulangan
`+`	1 atau lebih pengulangan
`?`	0 atau 1 pengulangan (opsional)
`\d`	Digit (0-9)
`\D`	Bukan digit
`\w`	Huruf, angka, underscore
`\W`	Bukan \w
`\s`	Whitespace (spasi, tab, dll)
`\S`	Bukan whitespace
`[abc]`	Salah satu dari a, b, atau c
`[^abc]`	Bukan a, b, atau c
`{n}`	Tepat n kali
`{n,m}`	Antara n dan m kali
`(pola)`	Grup / capture group
`a|b`	a atau b

5.3 Fungsi-Fungsi re

```
import re

teks = 'Nomor HP: 08123456789 dan 08987654321'

# re.search() - cari pertama kali
hasil = re.search(r'\d+', teks)
if hasil:
    print('Ditemukan:', hasil.group())
    print('Posisi:', hasil.span())

# re.findall() - cari semua kemunculan
semua_nomor = re.findall(r'\d+', teks)
print('Semua nomor:', semua_nomor)

# re.match() - cocokkan dari AWAL string
kalimat = 'Python sangat keren'
```

```
cocok = re.match(r'Python', kalimat)
print('Match:', cocok.group() if cocok else 'Tidak ditemukan')

# re.sub() - ganti pola dengan string baru
sensor = re.sub(r'\d', '*', teks)
print('Tersensor:', sensor)
```

Output:

```
Ditemukan: 08123456789
Posisi: (10, 21)
Semua nomor: ['08123456789', '08987654321']
Match: Python
Tersensor: Nomor HP: *********** dan ***********
```

5.4 Validasi dengan Regex

```
import re

# Validasi Email
def validasi_email(email):
    pola = r'^[\w.-]+@[\w.-]+\.[a-zA-Z]{2,}$'
    return bool(re.match(pola, email))

print(validasi_email('user@email.com'))    # True
print(validasi_email('bukan-email'))        # False
print(validasi_email('nama@domain.co.id'))  # True

# Validasi Nomor Telepon Indonesia
def validasi_hp(nomor):
    pola = r'^(08|\+628|628)\d{8,11}$'
    return bool(re.match(pola, nomor))

print(validasi_hp('081234567890'))   # True
print(validasi_hp('07123456789'))    # False
print(validasi_hp('+6281234567890')) # True
```

Output:

```
True
False
True
True
False
True
```

5.5 Grup dalam Regex

```
import re

# Ekstrak informasi dari teks
teks = 'Tanggal lahir: 15-04-2001'
pola = r'(\d{2})-(\d{2})-(\d{4})'
hasil = re.search(pola, teks)

if hasil:
    print('Tanggal penuh:', hasil.group(0))
    print('Hari:', hasil.group(1))
    print('Bulan:', hasil.group(2))
    print('Tahun:', hasil.group(3))

# Named groups (grup bernama)
pola2 = r'(?P<hari>\d{2})-(?P<bulan>\d{2})-(?P<tahun>\d{4})'
hasil2 = re.search(pola2, teks)
if hasil2:
    print('Hari (named):', hasil2.group('hari'))
    print('Tahun (named):', hasil2.group('tahun'))
```

Output:

```
Tanggal penuh: 15-04-2001
Hari: 15
Bulan: 04
Tahun: 2001
Hari (named): 15
Tahun (named): 2001
```

5.6 Flag Regex

```
import re

teks = 'Python PYTHON python PyThOn'

# re.IGNORECASE - abaikan besar/kecil huruf
hasil = re.findall(r'python', teks, re.IGNORECASE)
print('Case-insensitive:', hasil)

# re.MULTILINE - ^ dan $ untuk setiap baris
multi = '''baris pertama
baris kedua
baris ketiga'''

hasil2 = re.findall(r'^baris', multi, re.MULTILINE)
```

```
print('Awal setiap baris:', hasil2)

# re.DOTALL - . cocok dengan newline juga
teks2 = 'Awal\nTengah\nAkhir'
hasil3 = re.search(r'Awal.+Akhir', teks2, re.DOTALL)
print('DOTALL:', hasil3.group() if hasil3 else 'Tidak ditemukan')
```

Output:

```
Case-insensitive: ['Python', 'PYTHON', 'python', 'PyThOn']
Awal setiap baris: ['baris', 'baris', 'baris']
DOTALL: Awal
Tengah
Akhir
```

RINGKASAN & LATIHAN

Rangkuman Materi

- ✅ String adalah urutan karakter yang diapit tanda kutip (tunggal/ganda/triple).
- ✅ Indexing menggunakan angka (positif dari depan, negatif dari belakang).
- ✅ Slicing: teks[start:stop:step] — sangat berguna untuk mengambil bagian string.
- ✅ Metode seperti upper(), lower(), split(), join(), replace() sangat sering dipakai.
- ✅ f-string adalah cara terbaik dan modern untuk format string di Python.
- ✅ Parsing string = mengurai teks untuk mendapat informasi tertentu (split, strip, dll).
- ✅ Regex menggunakan modul re: search(), findall(), match(), sub() untuk pola teks kompleks.

Latihan Soal

1. Buatlah program yang menerima input nama lengkap dan menampilkan inisialnya. Contoh: "Budi Santoso" → "B.S."

2. Buat fungsi yang mengecek apakah sebuah kata adalah palindrom (dibaca sama dari depan/belakang).

3. Parsing string berikut untuk mengekstrak nama, kota, dan profesi: "Andi|Jakarta|Dokter"

4. Gunakan regex untuk mencari semua alamat email dalam sebuah paragraf teks.

5. Buat validator kata sandi yang mensyaratkan: minimal 8 karakter, ada huruf besar, ada angka.

🎯 Tips Belajar

1. Praktikkan setiap contoh kode di Python. 2. Gunakan situs regex101.com untuk uji coba regex secara interaktif. 3. Ingat: string di Python bersifat immutable (tidak bisa diubah langsung). 4. Gunakan help(str) di Python untuk melihat semua metode string.